

Deployment and operations

AI Failures Museum | Team Project COM2020 | Group Java The Hutt

System overview

The product is a Django 6 web application (museum project) with the exhibits app providing public pages, accounts, quizzes, comments, bookmarks, and role-based curator/admin features. Default database is SQLite (db.sqlite3 under backend/). Uploaded media and exhibit images use Django file storage (MEDIA_ROOT).

Prerequisites

Python 3.10 or newer; pip. Install dependencies from backend/requirements.txt (pinned full environment) or, for a minimal install, the root requirements.txt (Django and Pillow only). Pillow is required for exhibit images.

Local development startup

From the repository root:

```
cd backend
```

```
python -m venv venv
```

Windows: venv\Scripts\activate

macOS/Linux: source venv/bin/activate

```
pip install -r requirements.txt
```

(or: pip install -r ../requirements.txt from backend if using root minimal file)

```
python manage.py migrate
```

```
python manage.py seed_exhibits
```

```
python manage.py createsuperuser (optional; for Django admin)
```

```
python manage.py runserver
```

Open <http://127.0.0.1:8000/> . Django admin: <http://127.0.0.1:8000/admin/> .

Health check URL: <http://127.0.0.1:8000/health/> .

Seed data is loaded from backend/exhibits/data/seed/exhibits.json and quizzes.json. Images are resolved under backend/media/exhibits/images/ (see media README).

Dev container (optional)

A VS Code / GitHub Codespaces devcontainer is defined under .devcontainer/ using Python 3.11; post-create installs root requirements.txt. After opening the container, run migrate, seed, and runserver from backend/ as above.

Production and operations checklist

Before any public deployment: set DEBUG=False; set SECRET_KEY from environment or secrets manager (never commit production secrets); configure ALLOWED_HOSTS; serve over HTTPS; use a production-grade database if SQLite is insufficient; configure static files (collectstatic) and a proper web server (e.g. gunicorn + reverse proxy); plan MEDIA_ROOT on persistent storage and backups; schedule dependency and security updates; restrict Django admin exposure.

Day-to-day operations: create staff accounts via signup then promote admins with python manage.py set_admin <username>; curators manage exhibits and quizzes via in-app flows and Django admin as designed; monitor disk use for SQLite and media.